IITB Insti-Assist

Chosen Scope and Why

Out of the four scopes in the brief, I went with the Academic Assistant. It answers questions about IIT Bombay's official undergraduate academic rules — things like the academic calendar, registration and academic-standing categories, how grading and SPI/CPI are calculated, student medical entitlements, and medals or academic prizes.

I picked this over Hostel & Campus Life, Council/Club, or a general Insti Assistant mainly because academic rules at IIT Bombay are genuinely a pain to navigate — dense, clause-heavy, and spread across several separate PDFs on the Academic Office website. That's exactly the kind of situation where a student ends up guessing wrong or just giving up, and where RAG actually earns its keep: a precise, sourced answer beats a hallucinated guess by a mile. It's also the scope with the clearest "ground truth" — rules like the grading scale or the SPI formula have one official answer, which made it much easier to check whether the grounding and the "I don't know" fallback were actually working.

Data Sources Used

All five source documents are official PDFs from the IIT Bombay Academic Office (acad.iitb.ac.in), converted to Markdown before ingestion. Each file keeps a link back to its original PDF, which shows up in the app's sources panel so any answer can be traced back to where it actually came from:

- **Academic Calendar 2026–27** — semester dates, registration windows, exam schedules. acad.iitb.ac.in/.../Academic_Calendar_2026-27_FINAL.pdf
- **UG Rules & Regulations — Curriculum & Registration** — credit structure, Minor/Honours, registration procedure, academic-standing categories I–VI. acad.iitb.ac.in/.../UG_RULE_BOOK.pdf
- **UG Rules & Regulations — Examination & Grading** — grading scale, SPI/CPI formulas, transcript rules, ARP, exit-degree, branch change. *(Same source PDF as above, split by topic — see Chunking Strategy.)*
- **Student Medical Rules** — hospital entitlement, reimbursement ceilings, medical advance process. acad.iitb.ac.in/.../student-medical-rules.pdf
- **Rules for Award of Medals and Academic Prizes** — President of India Medal, Institute Gold/Silver Medal, academic prize eligibility. acad.iitb.ac.in/.../RulesforAwardofMedalsandAcademicprizesforUGandPG.pdf

Chunking Strategy and Why

I split documents on blank lines into paragraphs, then greedily pack them into roughly 600-character chunks, carrying 150 characters of overlap into the next chunk so a rule that straddles a boundary doesn't get cut in half. I kept the chunk size smaller than the usual 800-character default because this isn't course-notes prose — it's dense rulebook text. A single grading category, a medal's eligibility criteria, or a specific registration category (like "Category III") needs to stay intact and not get diluted by whatever clause happens to sit next to it, or the retrieval similarity scores stop meaning much.

The UG Rule Book PDF itself is over 40 pages and covers curriculum, registration, examinations, grading, and performance requirements all together. Rather than ingest it as one big document, I split it into two topic-coherent Markdown files — Curriculum & Registration, and Examination & Grading. That keeps the paragraphs in each file topically clustered, which helps the embedding model tell apart, say, a question about registration load versus one about grading, instead of both fighting it out against chunks pulled from one giant undifferentiated file.

For embeddings I used sentence-transformers/all-MiniLM-L6-v2 (384 dimensions), indexed with a FAISS flat inner-product index over L2-normalised vectors — which works out to cosine similarity. At query time it pulls the top-k chunks (4 by default), and a similarity threshold on the top match gates a three-tier groundedness label: Strongly, Weakly, or Not grounded. That label also decides whether the LLM even gets called — if it comes back "Not grounded," the app short-circuits straight to "I don't know" and skips the API call entirely.

LLM Integration

The retrieved chunks get dropped into a prompt, with bracketed source labels, and sent to the Google Gemini API (gemini-flash-latest). The system prompt tells the model to answer only from the given context and to say "I don't know" when the context doesn't back up an answer. I went with Gemini over Claude or OpenAI mainly for practical reasons — Google AI Studio has a genuinely free tier with no credit card and a daily-reset quota, which meant I could run the whole project at zero cost.

Known Limitations / What I'd Improve With More Time

- Top-k is fixed no matter how complex the query is — a multi-part question would really benefit from adaptive k.
- There's no memory across turns; every query is answered in isolation, with no awareness of what was asked before.
- You can't upload a document at query time — the knowledge base is locked in at ingestion, though the ingestion pipeline itself can take in additional PDFs if you re-run it.
- Academic rules and dates shift every year (a new Academic Calendar, for instance), so the knowledge base needs periodic re-ingestion from the latest PDFs on acad.iitb.ac.in to stay accurate.
- The groundedness label is just a similarity threshold, not a properly calibrated confidence score, so it can occasionally mislabel a borderline match.
- The app depends on a live Gemini API key and an internet connection — there's no offline fallback — and the free tier's rate limits would need upgrading for heavier concurrent use.

Full source code, setup instructions, and data files are in the accompanying GitHub repository README.